

Redes de Comunicaciones y Cómputo Distribuido

2do Cuatrimestre 2026

Taller - Learning Bridge

1 Implementación y análisis con Containernet + Open vSwitch

2 Enunciado del taller

En este laboratorio se propone estudiar experimentalmente el siguiente problema:

¿Cómo aprenden los bridges la ubicación de los hosts en una red Ethernet y cómo cambia el comportamiento de reenvío de tramas a medida que se construyen las tablas de direcciones MAC?

Para comprender lo anterior, se deberá:

- Implementar la topología de red vista en la práctica 2 (ejercicio 2) utilizando Containernet y Open vSwitch.
- Analizar el estado inicial de las tablas de direcciones MAC de cada bridge antes de generar tráfico de red.
- Generar tráfico entre hosts y analizar cambios en las tablas de direcciones MAC.
- Observar el movimiento de tramas dentro de la red mediante capturas de tráfico.
- Analizar las tablas de forwarding en los bridges para entender el proceso de aprendizaje MAC.
- Insertar fallas en el enlace y analizar los cambios inducidos (reconstrucción de aprendizaje de direcciones MAC).

3 Propósito general del taller

El objetivo de este taller es observar **cómo se mueven tramas en una red L2**, cómo aparece el flooding inicial y cómo el *learning* MAC transforma la red en forwarding selectivo.

El taller utiliza Containernet (Mininet + Docker) y Open vSwitch.

4 Requisitos del sistema

El taller se ejecuta completamente dentro de un contenedor Docker, pero requiere:

- Linux (Ubuntu 22.04/24.04)
- Docker
- Permisos para ejecutar contenedores privilegiados

No es necesario instalar Mininet ni Open vSwitch.

5 Construcción del contenedor del taller

Desde el directorio raíz del taller (**Taller_Learning_Bridge_Containernet/**):

```
docker build -t lb-containernet:lab -f docker/Dockerfile .
```

Este paso construye una imagen que contiene:

- Containernet
- Mininet
- Open vSwitch
- Python 3
- Librerías necesarias para automatización y análisis

6 Ejecución del contenedor

Ejecutar el contenedor con privilegios:

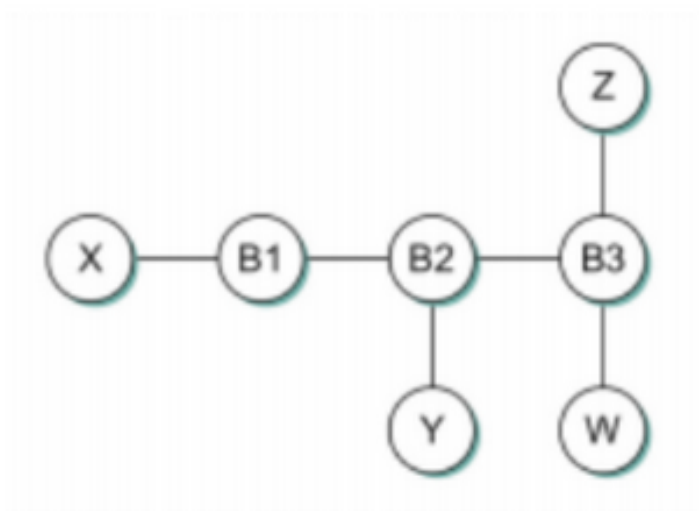
```
docker run --rm -it \  
  --privileged --name taller-lb \  
  -v /var/run/docker.sock:/var/run/docker.sock \  
  -v /lib/modules:/lib/modules \  
  -v $(pwd):/workshop \  
  lb-containernet:lab
```

Dentro del contenedor, el directorio del taller estará disponible en:

```
/workshop
```

7 Construcción de la topología

La figura a continuación muestra la topología a usar en el taller. Observe que esta topología fue la analizada en la práctica anterior (Learning Bridge).



Una vez dentro del directorio del taller ejecute:

```
cd /workshop
python3 topo_learning_bridge.py
```

Este script crea:

- 4 hosts (X, Y, Z y W)
- 3 Learning Bridge (B1 a B3)

Al finalizar, se ingresa automáticamente a la CLI de Containernet.

7.1 Objetivo de este paso

Construir una red con Learning Bridges de Capa 2.

8 Verificación inicial de la topología

Una vez en la CLI de Containernet puede ejecutar:

```
nodes
net
```

Deben obtener algo como:

Para **nodes**:

```
available nodes are:
B1 B2 B3 W X Y Z
```

Para **net**:

```
X X-eth0:B1-eth1
Y Y-eth0:B2-eth1
Z Z-eth0:B3-eth1
W W-eth0:B3-eth2
B1 lo: B1-eth1:X-eth0 B1-eth2:B2-eth2
B2 lo: B2-eth1:Y-eth0 B2-eth2:B1-eth2 B2-eth3:B3-eth3
B3 lo: B3-eth1:Z-eth0 B3-eth2:W-eth0 B3-eth3:B2-eth3
```

Esto permite verificar:

- Qué nodos existen.
- Cómo están conectados.
- Qué interfaces tiene cada bridge.

9 Observabilidad del estado inicial de las tablas MAC

Antes de generar tráfico, puede verificar que no existan MAC dinámicas aprendidas usando estos comandos desde la CLI de Containernet, para cada bridge:

```
sh ovs-appctl fdb/show B1
sh ovs-appctl fdb/show B2
sh ovs-appctl fdb/show B3
```

Después de ejecutar los comandos anteriores para cada bridge, no debería ver todavía claramente MAC de los hosts (X, Y, Z y W) aprendidas por actividad reciente, por ejemplo:

port	VLAN	MAC	Age
------	------	-----	-----

10 Provocar aprendizaje de direcciones MAC y observar cambios

10.1 Primer envío (evento flooding + learning)

Desde otra terminal dentro del mismo contenedor ejecute:

```
tcpdump -i B2-eth1 -nn
```

Nota: se puede abrir otra terminal dentro del contenedor ejecutando:

```
docker exec -ti taller-lb bash
```

Esto permite observar generación de ARP inicial, tráfico y aprendizaje de MAC origen, por ejemplo:

```
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on B2-eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:07:00.492324 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 37864, seq 1, length 64
13:07:00.492346 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 37864, seq 1, length 64
13:07:05.640063 ARP, Request who-has 10.0.0.1 tell 10.0.0.2, length 28
13:07:05.640417 ARP, Request who-has 10.0.0.2 tell 10.0.0.1, length 28
13:07:05.640431 ARP, Reply 10.0.0.2 is-at 02:a6:da:58:0b:07, length 28
13:07:05.640514 ARP, Reply 10.0.0.1 is-at d2:6c:92:05:b0:f7, length 28
13:07:57.351871 IP6 fe80::505f:55ff:fe7f:237b > ff02::2: ICMP6, router solicitation, length 16
```

Lo anterior lo puede observar una vez que ejecuta desde la CLI de Containernet:

```
X ping -c 1 10.0.0.2
```

Alternativamente, se puede usar la herramienta Wireshark para visualizar las tramas capturadas en vez de verlas en la terminal. Para eso pueden ejecutar en una terminal nueva (fuera del contenedor):

```
docker exec taller-lb tcpdump -nl -s 2048 -w - -i B1-eth1 | wireshark -k -i -
```

10.2 Confirmar aprendizaje de direcciones MAC

Inmediatamente después de realizar el ping anterior puede volver a ejecutar para cada bridge:

```
sh ovs-appctl fdb/show B1
sh ovs-appctl fdb/show B2
sh ovs-appctl fdb/show B3
```

Debería ver algo como esto:

```
containernet> sh ovs-appctl fdb/show B1
port  VLAN  MAC                      Age
  1      0  d2:6c:92:05:b0:f7           1
  2      0  02:a6:da:58:0b:07           1
containernet> sh ovs-appctl fdb/show B2
port  VLAN  MAC                      Age
  2      0  d2:6c:92:05:b0:f7          13
  1      0  02:a6:da:58:0b:07          12
containernet> sh ovs-appctl fdb/show B3
port  VLAN  MAC                      Age
  3      0  d2:6c:92:05:b0:f7          22
```

Observe que:

- Aparecen MAC dinámicas con Age.
- Cada bridge aprende lo que observa localmente.
- El puerto asociado indica por dónde cree el bridge que vive esa MAC.

10.3 Segundo envío (forwarding selectivo)

Ahora puede volver a ejecutar:

```
X ping -c 3 10.0.0.2
```

Luego de realizar el ping podemos ver desde la otra terminal donde estamos realizando la observabilidad del proceso con `tcpdump -i B2-eth1 -nn` (o usando Wireshark) lo siguiente:

```
13:33:04.986023 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 4987, seq 1, length 64
13:33:04.986049 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 4987, seq 1, length 64
13:33:06.023902 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 4987, seq 2, length 64
13:33:06.023920 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 4987, seq 2, length 64
13:33:07.047948 IP 10.0.0.1 > 10.0.0.2: ICMP echo request, id 4987, seq 3, length 64
13:33:07.047966 IP 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 4987, seq 3, length 64
13:33:10.311861 ARP, Request who-has 10.0.0.1 tell 10.0.0.2, length 28
13:33:10.312112 ARP, Request who-has 10.0.0.2 tell 10.0.0.1, length 28
13:33:10.312119 ARP, Reply 10.0.0.2 is-at 02:a6:da:58:0b:07, length 28
13:33:10.312127 ARP, Reply 10.0.0.1 is-at d2:6c:92:05:b0:f7, length 28
13:33:37.448136 IP6 fe80::1030:56ff:fe19:efca > ff02::2: ICMP6, router solicitation, length 16
```

Observe que:

- Ya no es necesario inundar toda la red.
- El forwarding se vuelve selectivo porque el destino está en la tabla de direcciones MAC.

11 Experimento de falla controlada de enlace y reconstrucción de aprendizaje

El objetivo de este experimento es inyectar una falla y observar qué cambia. La idea es simular un cable desconectado entre bridges para forzar:

- Pérdida de conectividad parcial (o degradación).
- Necesidad de re-aprender MAC.
- Reaparición de flooding en el rearmado.

Primero podemos elegir un enlace entre bridges (ver paso 6, al ejecutar `net`), esto nos permite encontrar una interfaz que conecte bridges (por ejemplo B2-eth2).

Luego podemos bajar el enlace ejecutando:

```
sh ip link set B2-eth2 down
```

Pruebe conectividad ejecutando de vuelta:

```
X ping -c 3 10.0.0.2
```

Debería ver algo como:

```
containernet> X ping -c 3 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2086ms
```

Observe que se pierde conectividad.

Luego puede volver a observar si hay cambios en las tablas de direcciones MAC de cada bridge, ejecutando:

```
sh ovs-appctl fdb/show B1
sh ovs-appctl fdb/show B2
sh ovs-appctl fdb/show B3
```

Ahora restaure el enlace ejecutando:

```
sh ip link set B2-eth2 up
```

y vuelva a generar tráfico y observe el reaprendizaje ejecutando:

```
X ping -c 3 10.0.0.2
sh ovs-appctl fdb/show B2
```

12 Instrumentación automática del aprendizaje MAC

Objetivo de la instrumentación: registrar evidencia del tamaño de la tabla de direcciones MAC por bridge (qué MAC aparece, en qué puerto y en qué momento) y cómo cambia el estado cuando generamos tráfico y cuando hay fallas.

Antes de iniciar la instrumentación cree la carpeta **results** dentro del directorio de trabajo:

```
mkdir -p results
```

Nota: Pueden usar este comando para borrar las direcciones MAC de todos los bridges:

```
sh ovs-appctl fdb/flush B1
sh ovs-appctl fdb/flush B2
sh ovs-appctl fdb/flush B3
```

Luego en otra terminal dentro del contenedor ejecute durante 60 segundos:

```
python3 fdb_monitor.py --bridges B1 B2 B3 --interval 1 --duration 60 --outdir results
```

Debe obtener dentro de la carpeta **results** el archivo **results/fdb_monitor.csv**, con un contenido como por ejemplo:

```
timestamp,bridge,entries_count,mac,port,vlan,age,note
2026-02-19T14:37:09,B1,1,aa:78:9e:56:df:1e,2,0,129,
2026-02-19T14:37:09,B2,0,,,,no_entries
2026-02-19T14:37:09,B3,0,,,,no_entries
2026-02-19T14:37:10,B1,1,aa:78:9e:56:df:1e,2,0,130,
2026-02-19T14:37:10,B2,0,,,,no_entries
2026-02-19T14:37:10,B3,0,,,,no_entries
2026-02-19T14:37:11,B1,1,aa:78:9e:56:df:1e,2,0,131,
2026-02-19T14:37:11,B2,0,,,,no_entries
2026-02-19T14:37:11,B3,0,,,,no_entries
```

En el archivo CSV cada columna representa lo siguiente:

- **timestamp**: Momento de la medición.
- **bridge**: Bridge observado (B1, B2, B3).
- **entries_count**: Cantidad total de MAC aprendidas en ese instante.
- **mac**: Dirección MAC observada.
- **port**: Puerto donde fue aprendida.
- **vlan**: VLAN (si aplica).
- **age**: Tiempo desde el último uso.
- **note**: Indicador de error o vacío.

Este experimento mide el proceso dinámico de aprendizaje MAC en bridges OVS (evoluciona el conocimiento del bridge sobre la red).

12.1 Analizar el impacto de una falla

Si durante los 60 segundos de duración del experimento ejecuta:

```
sh ip link set B2-eth2 down
```

Podemos observar en el CSV:

- MAC que desaparecen.
- MAC que cambian de puerto.
- Reaparición de entradas tras reconvergencia.

Lo cual demuestra que el aprendizaje depende de la topología activa.

13 Conclusiones

El taller demuestra que:

- Un learning bridge no tiene inteligencia global.
- Opera localmente, en base a observación.
- Construye conocimiento progresivo.
- Optimiza forwarding sin necesidad de configuración manual.
- Este comportamiento emergente es un ejemplo clásico de sistema distribuido basado en información local.

14 Referencias bibliográficas

- Larry Peterson and Bruce Davie, *Computer Networks: A Systems Approach*, Elsevier, 2022. <https://github.com/SystemsApproach/book>. Licencia: CC BY 4.0.
- Maarten van Steen and Andrew S. Tanenbaum, *Distributed Systems: Principles and Paradigms*, 4th Edition, Version 01, 2023. <https://www.distributed-systems.net/index.php/books/ds4/>.
- M. Peuster, H. Karl, and S. v. Rossem, *MeDICINE: Rapid Prototyping of Production-Ready Network Services in Multi-PoP Environments*. IEEE NFV-SDN, Palo Alto, USA, pp. 148–153, 2016. DOI: <https://doi.org/10.1109/NFV-SDN.2016.7919490>.